
INFO 1112 — Computing 1B OS and Network Platforms

Assignment: 1

Due Date: September 20, 2026, 11:59 PM

Version: 1.0

Consider a (not very practical) hypothetical computer CPU, with

- 16 bits (2 Bytes) long instructions
- memory capacity of 256 bytes
- has 4 registers.

Each instruction (size of 2 Bytes) has the following format:

- Starts with an Instruction Code/Opcode (6 bits)
- Followed by a Register number information (2 bits), and
- Lastly a Memory Address (8 bits).

Opcode (6 bits)	Which Register? (2 bits)	Memory Address (8 bits)
-----------------	--------------------------	-------------------------

Figure 1: Instruction format

The instructions of the CPU and their opcodes are:

Instruction Name	Instruction Code/ Opcode (6 bits)	Register (2 bits)	Memory Address (8 bits)
LOAD	1 (000001)	reg. address	memory address
STORE	2 (000010)	reg. address	memory address
ADD	3 (000011)	reg. address	memory address
SUB	4 (000100)	reg. address	memory address
QUIT	8 (001000)	00	00000000
PRINT	9 (001001)	reg. address	00000000

Figure 2: Instructions and their opcodes

More details about these operations are as follows.

LOAD

Transfers data from a given memory address (M) to a given register (R).

$$\text{Register} \leftarrow \text{Memory}[\text{Address}]$$

That means after a successful LOAD operation, same data would be in both M and R.

STORE

Transfers data from a given register (R) to a given memory address (M).

$$\text{Memory}[\text{Address}] \leftarrow \text{Register}$$

That means after a successful STORE operation, same data would be in both R and M.

ADD

Performs addition of data stored in register (R) with data stored in memory address (M).

$$\text{Register} \leftarrow \text{Register} + \text{Memory}[\text{Address}]$$

That means after a successful ADD operation, the result would be stored in R.

SUB

Performs subtraction of data stored in register (R) from the data stored in memory address (M).

$$\text{Register} \leftarrow \text{Register} - \text{Memory}[\text{Address}]$$

That means after a successful SUB operation, the result would be stored in R. *Since, this computer is a very basic one, therefore it can only perform a SUB operation if the data in R is $\geq$ the data in M.* Otherwise, an error message is generated and the contents in R and M remains unchanged.

QUIT

Works as a sentinel to indicate that after this instruction, the given program does not contain any further instruction.

PRINT

Displays the content of a specified register in STDOUT.

At the beginning, the following (initialized) state should be present:

- an array of 256 elements representing the *memory*, should contain 0s in them.
- another array of 4 elements representing the *registers*, should contain 0s in them.
- the program counter, which holds the address of the next instruction need be executed, is at N i.e., our first instruction to be executed is located at memory address N (8 bits) and address $(N + 1)$ (8 bits), since instructions are 16 bits long.

Program Structure We will write our programs in .vsc (very-simple-computer) files by using the following layout.

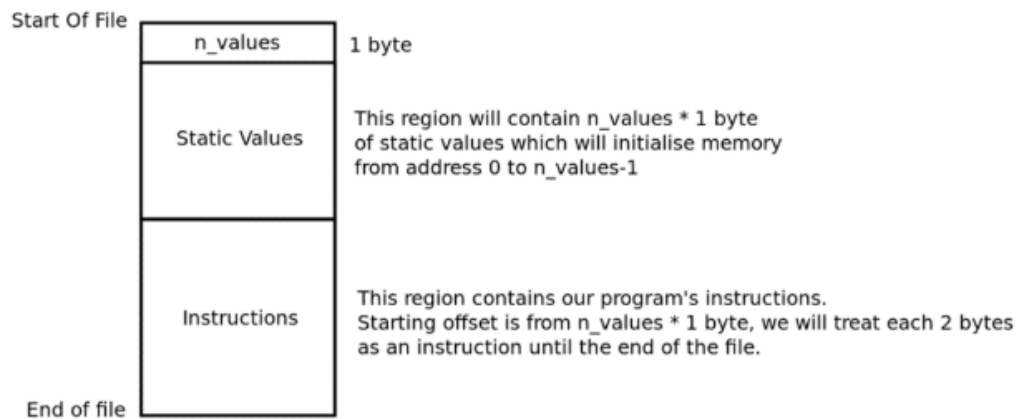


Figure 3: Layout of the .vsc files

An example of a .vsc file is given below.

```

prog1.vsc
1 2
2 20
3 40
4 LOAD,0,0
5 ADD,0,1
6 STORE,0,14
7 PRINT,0,0
8 QUIT,0,0
9

```

Figure 4: A sample .vsc file

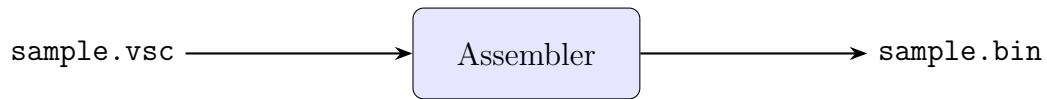
This program should be *interpreted* as such:

- 2 values are provided which are 20 and 40. And they are in memory address 0 and memory address 1 respectively.

- Then 20 is loaded (from address 0) into register 0.
- After that, the content of register 0, which is 20, is added with the content of memory address 1, which is 40. The result of this operation i.e., 60 is still in register 0 (due to the nature of the ADD operation).
- Then 60 is transferred from register 0 to memory address 14 using the STORE operation.
- The program then asks to display the content of register 0. So 60 gets displayed to STDOUT.
- Finally “seeing” QUIT, the interpreter understands that no further instruction is incoming.

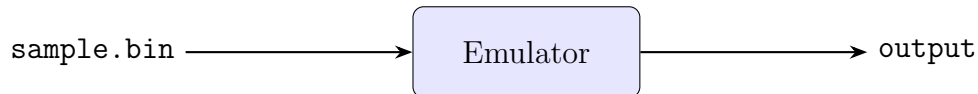
Requirements

1. Design an Assembler program, *assembler.sh*, which will convert a given .vsc file into binary equivalent (.bin) file.



```
1 $ bash assembler.sh sample.vsc
```

2. Design an Emulator program, *emulator.sh*, that models the hypothetical VSC. It takes a .bin file as input and executes/carries out the (binary) instructions contained in that .bin file.



```
1 $ bash emulator.sh sample.bin
```

3. Design test appropriate test cases.
4. After testing, containerize your VSC (with all the dependencies) so that your tutor/marker can test it.
5. **A compulsory component:** After the deadline, meet with your tutor/marker for a discussion session during the tutorial hours. **Without this, a grade of 0 would be awarded.**